



1

Building the UI for Your Messaging App

In this first chapter, we're going to start building a messaging app called WhatsPackt (referring to a popular messaging app that you probably already know about). At this point in the project, we must make some important technical decisions and create the structure needed to build it. This is what we will be focusing on, as well as working on the app's user interface.

By the end of this chapter, you will have hands-on experience creating a messaging app from scratch, organizing and defining the app modules, deciding which dependency injection framework you will use, using Jetpack Navigation to navigate between the app features, and using Jetpack Compose to build the main parts of the user interface.

This chapter is organized into the following topics:

- Defining the app structure and navigation
- Building the main screen
- Building the chats list
- Building the messages list

Technical requirements

Android Studio is the official standard **integrated development environment (IDE)** for developing Android apps. Although you can use other IDEs, editors, and Android tools if you prefer, all the examples in this book will be based on this IDE.

For that reason, we recommend that you set up your computer with the latest stable version of Android Studio installed. If you haven't already, you can download it here:

<https://developer.android.com/studio>. By following the installation steps, you will be able to install the IDE and set up at least one emulator with one Android SDK installed.

Once installed, we can start creating the project. Android Studio will offer us a set of templates to start with. We will choose the **Empty Activity** option, as shown in the following screenshot:

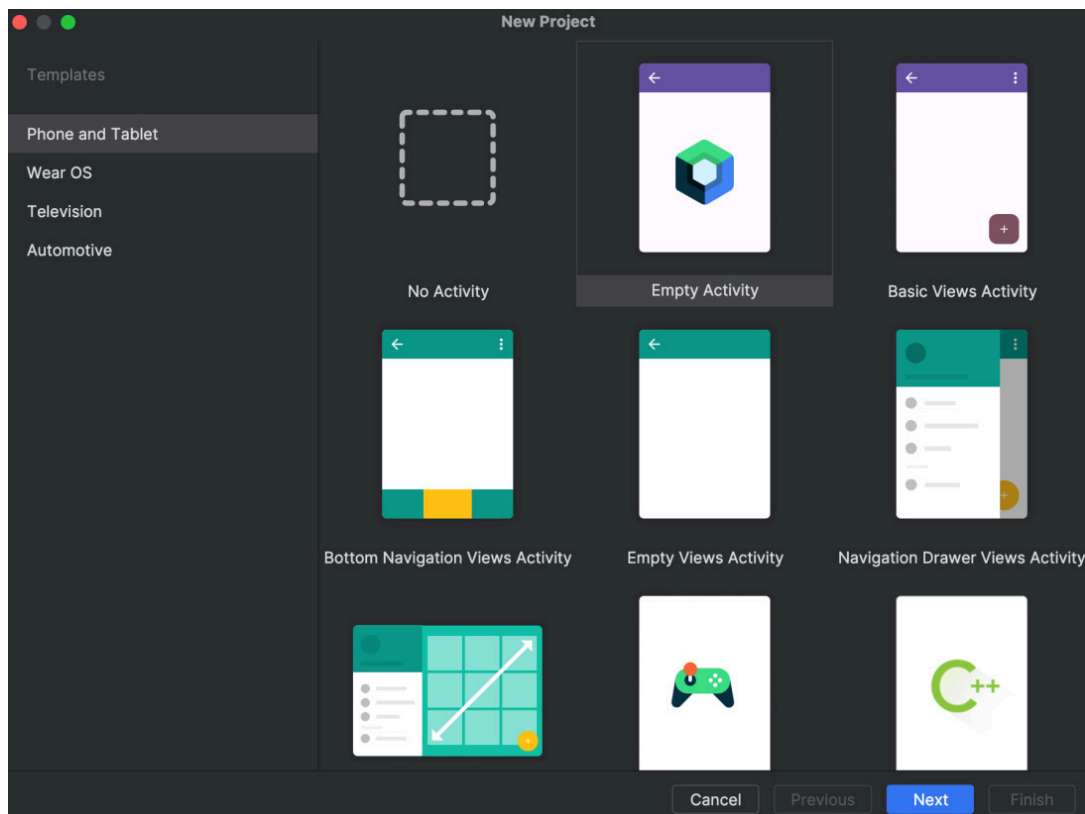


Figure 1.1: Android Studio new project template selection with the Empty Activity option selected

You will then be asked to select a project and package name:

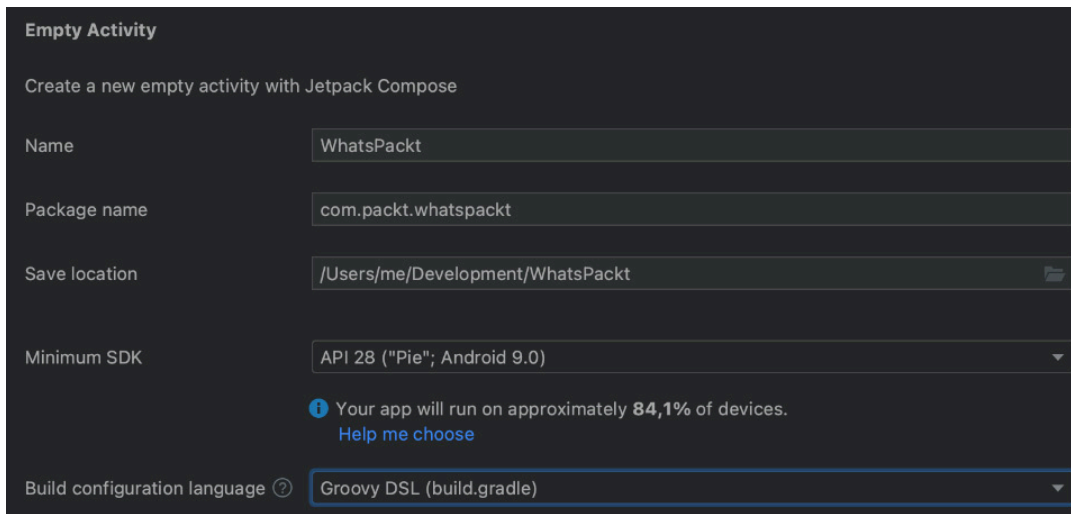


Figure 1.2: Android Studio – adding a new project name and package name

After that, you're all set! Android Studio will generate the main folders and files needed so that you can start working on our project. Your project structure should look as follows:

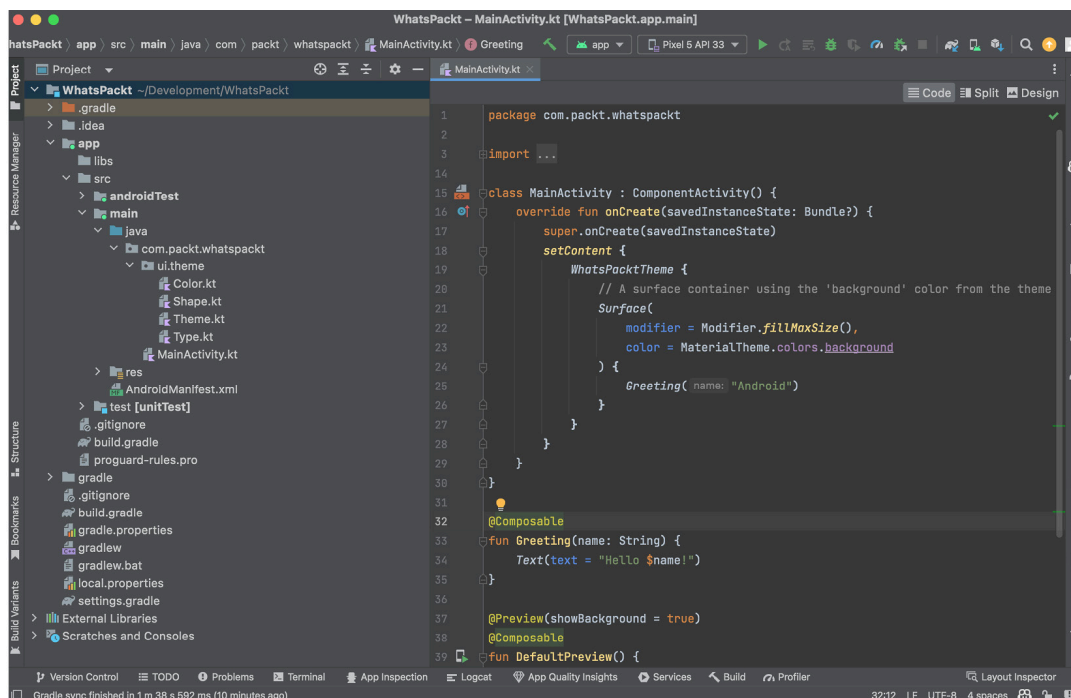


Figure 1.3: Android Studio – project template structure

Note that all the code for this chapter can be found in this book's GitHub repository: <https://github.com/PacktPublishing/Thriving-in-Android-Development-using-Kotlin/tree/main/Chapter-1/WhatsPackt>.

Now, we are ready to start coding our new messaging app. To do so, we will have to make some important technical decisions: we will have to decide how our project is going to be structured, how we will navigate between the different screens or features, and how we are going to set and provide the components needed (defining and organizing the dependencies between every component).

Defining the app structure and navigation

Before designing the app structure, we must have a basic idea of the features it should include. In our case, we want to have the following:

- A main screen to create new or access already existing conversations
- A list containing all the conversations
- A screen for a single conversation

As this is going to be a production-ready app, we must design its code base while considering that it should be easy to scale and maintain. In that regard, we should use modularization.

Modularization

Modularization is the practice of dividing the code of an application into loosely coupled and self-contained parts, each of which can be compiled and tested in isolation. This technique allows developers to break down large and complex applications into more manageable parts that are easier to maintain.

By modularizing Android applications, modules can be built in parallel, which can significantly improve build time. Additionally, independent modules can be tested separately, which makes it easier to identify and correct errors.

While the most common way to create modules in Android development is by utilizing the Android libraries system via Gradle dependencies, alternative build systems such as Bazel and Buck also facilitate modularization. Bazel provides a robust system for declaring modules and dependencies, and its parallelized building capabilities can lead to even faster build times. Similarly, Buck also supports modular development by providing fine-grained build rules and speeding up incremental builds.

By exploring various build systems, such as Gradle, Bazel, and Buck, developers can find the most suitable modular approach to structure their Android applications. Each build system offers unique features for managing dependencies and organizing code, enabling developers to implement various patterns to achieve a modular architecture.

Among the organizational patterns, the most common ones are modularization by layers and modularization by feature modules.

Modularization by layers

It is common to structure an app by grouping its components based on a set of layers depending on the architecture chosen by the developers. One popular architecture is clean architecture, which splits the code base between the data, domain (or business), and presentation layers.

With this approach, each module focuses on a specific layer of the architecture, such as the presentation layer, domain layer, or data layer. These modules are usually more independent of each other and may have different responsibilities and technologies, depending on the layer they belong to. Following this pattern, our app structure would look like this:

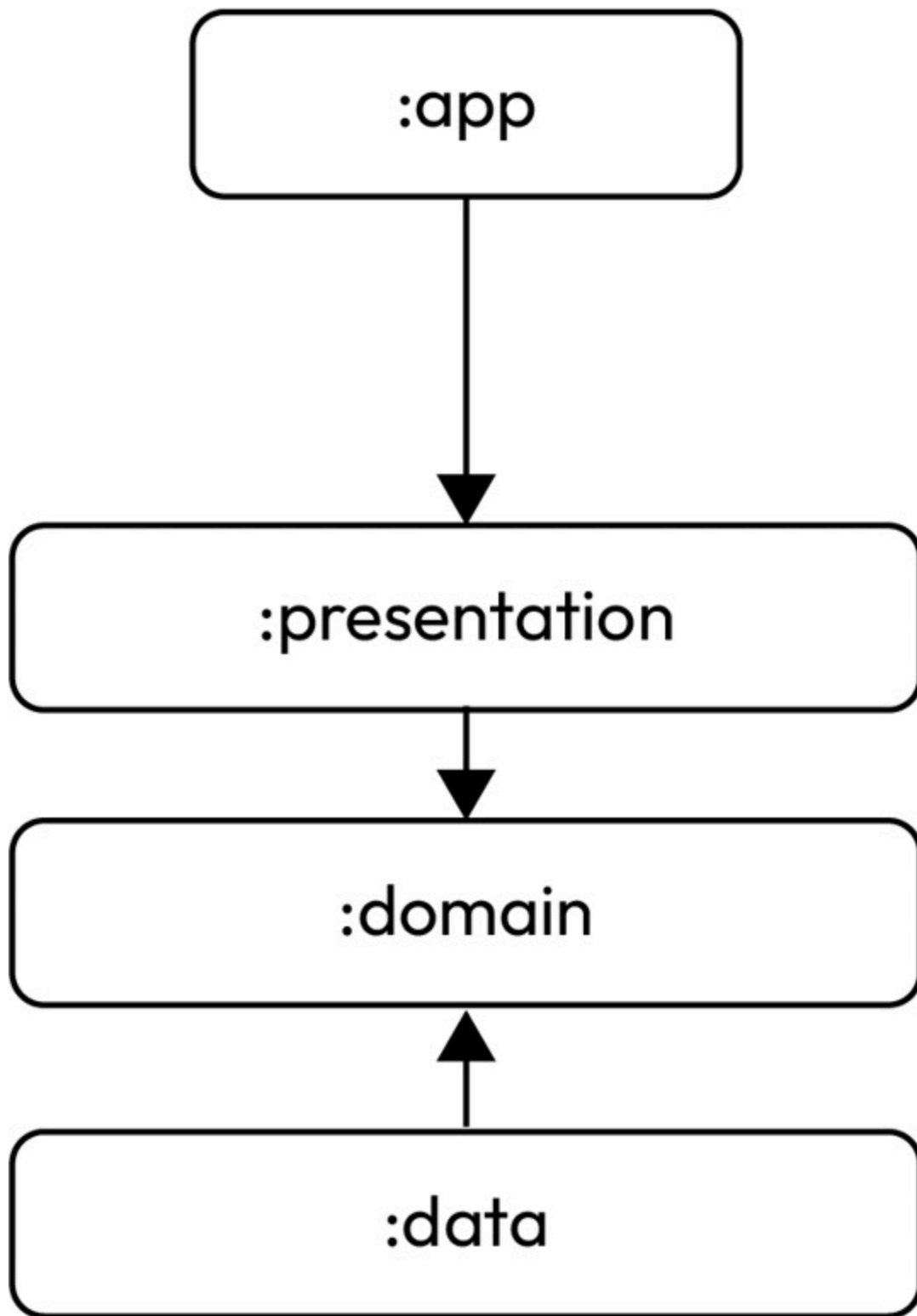


Figure 1.4: App modularization by layers

From this diagram, you can see why layer modularization is also referred to as vertical modularization.

Modularization by feature

When modularizing an app by feature (or using horizontal modularization), the application is divided into modules that focus on specific features or related tasks, such as authentication or navigation. These horizontal modules can share common components and resources. We can see this structure in the following figure:

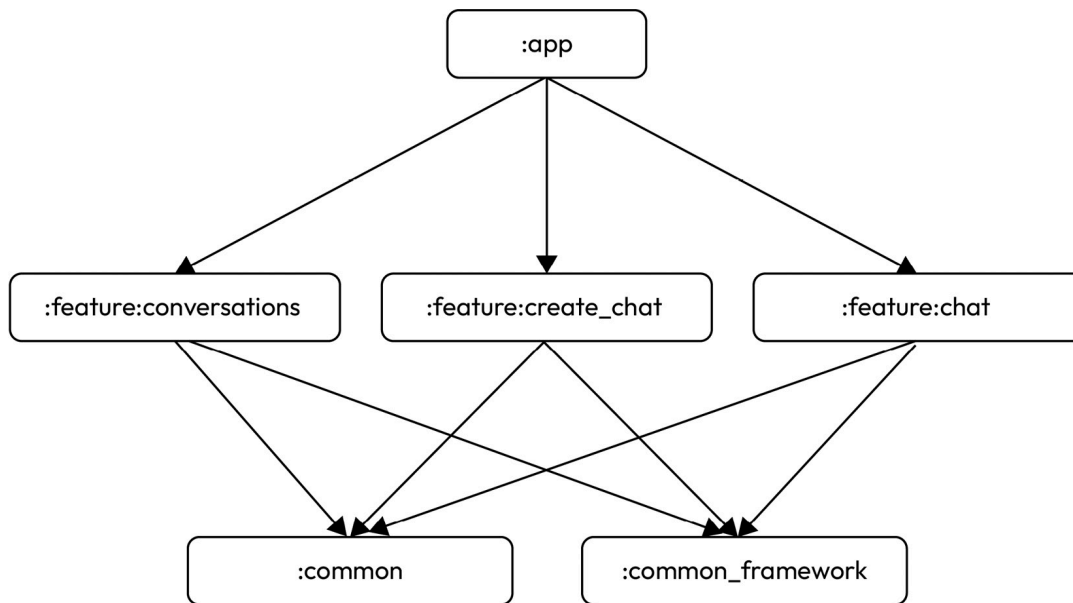


Figure 1.5: App modularization by feature

In our case, we are going to have a main **app** module that will depend on every one of the feature modules that our app needs (one for every one of the features we are going to implement). Then, every one of the feature modules will also depend on two other common modules (in this example, we have divided them into **common** and **common_framework**, using the first to include framework-independent code, and the second to use code that depends on the Android framework).

One of the main advantages of this pattern is that it can scale with the company if it evolves into a feature-based team (where every team is focused on a single or group of features). This will enable every team to be responsible for one feature module, or a set of feature modules, where they have ownership of the code in those modules. It also allows teams to be easily autonomous regarding their problem space and features.

WhatsPackt modularization

In our WhatsPackt example, we are going to combine both modularization approaches:

- We will use a modularization based on features for our features.
- We will use a modularization based on layers for the common modules. This will allow us to share common code between the feature modules.

The structure of our modules and its dependencies will be as follows:

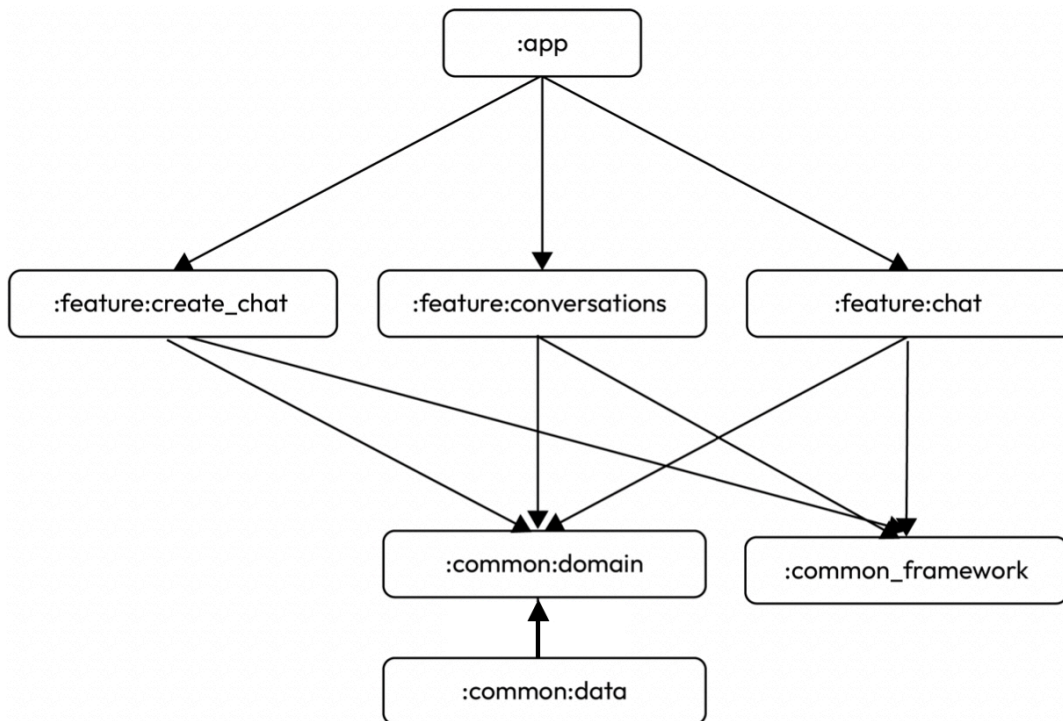


Figure 1.6: Our app modules structure and dependencies

Now, we are going to start creating this structure in Android Studio. To create a module, follow these steps:

1. Select **File** | **New...** | **New Module**.
2. In the **Create New Module** dialog, choose the **Android Library** template.
3. Fill in the **Module name**, **Package name**, and **Language** fields, as shown here:

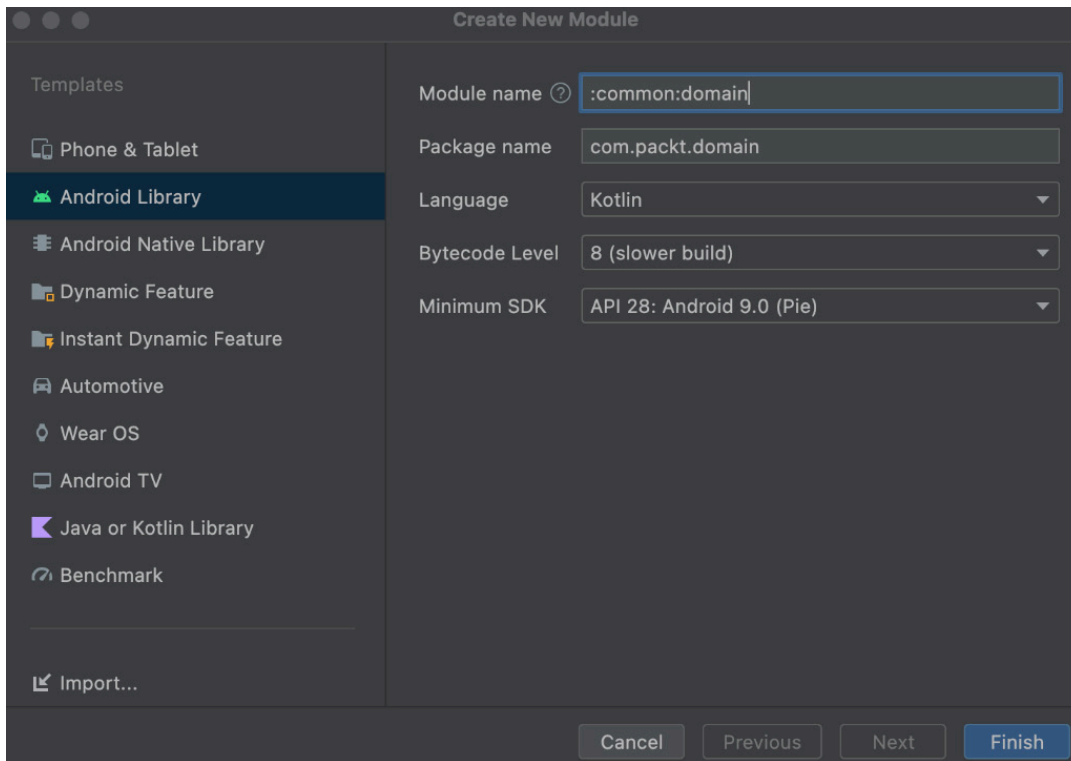


Figure 1.7: The Create New Module dialog

4. Click **Finish**.

We will have to do this same process for all the modules we want to build, except for the **:app** module, which should have been already created when we created the project. This is going to be our main point of entry to the app. So, we must create the following modules:

- **:common:domain**
- **:common:data**
- **:common:framework**
- **:feature:create_chat**
- **:feature:conversations**
- **:feature:chat**

Once we've done this, we should have built the following project structure:

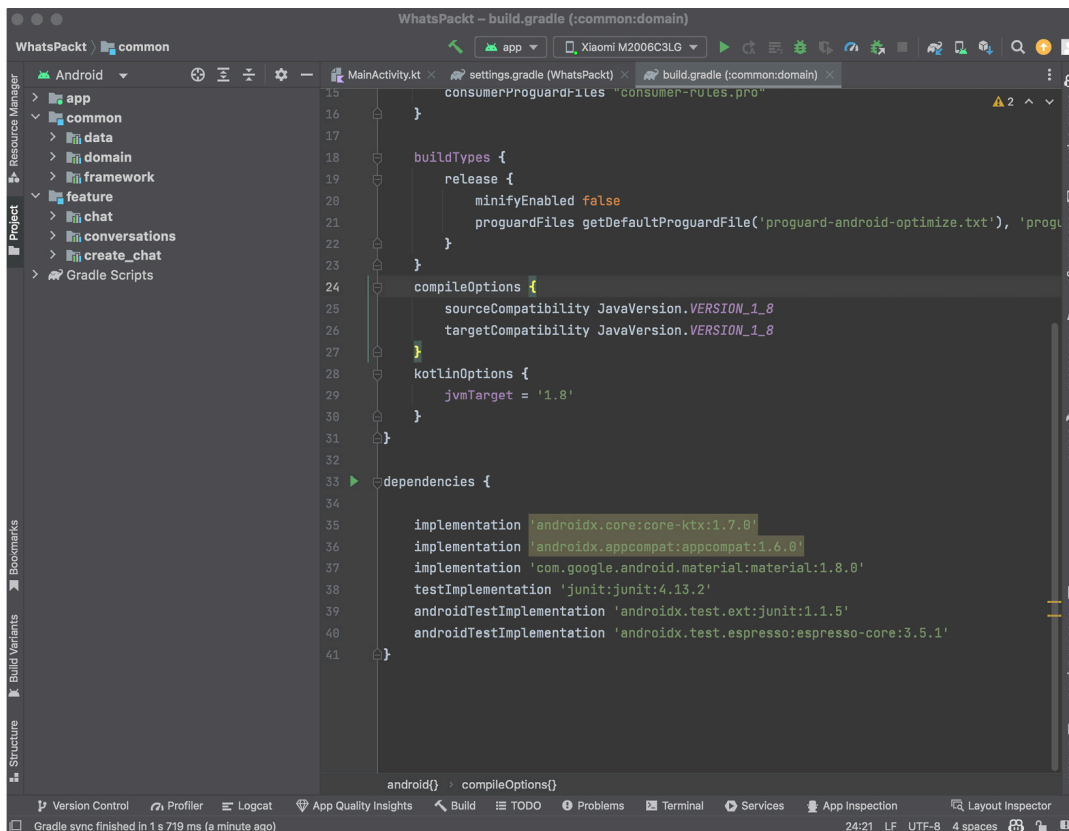


Figure 1.8: Project structure, including all modules

The next step is to set the dependencies between modules. We will do this in the **build.gradle** file of every module. For example, in the **build.gradle** file of the **:app** module, include the following code in the **dependencies** section:

```
dependencies {
    implementation project(':feature:chat')
    implementation project(':feature:conversations')
    implementation project(':feature:create_chat')
    // The rest of dependencies
}
```

Now that our app modules are ready, we can start working on the next step: dependency injection.

Dependency injection

Dependency injection is a design pattern and technique that's used in software engineering to decouple the objects in an application and reduce dependencies between them. In Android, dependency injection involves providing an instance of a class or a component to another class, rather than creating it explicitly within the class itself.

By implementing dependency injection in an Android app, you can make the app's code more modular, reusable, and testable. Dependency injection also helps improve the maintainability of the code base and reduce the complexity of the application architecture.

Some of the most popular dependency injection libraries that are used in Android development are as follows:

- **Dagger** (<https://dagger.dev/>): Dagger is a compile-time dependency injection library developed by Google that uses annotations and code generation to create a dependency graph that can be used to provide dependencies to the app's components. Its main advantage is that it builds this dependency graph at compile time, whereas other libraries (such as Koin) do it at runtime. For larger apps, this can imply a performance problem.
- **Hilt** (<https://dagger.dev/hilt/>): Hilt is a dependency injection library built on top of Dagger that provides a simplified way to perform dependency injection in Android apps. It reduces the boilerplate code required for Dagger and provides predefined bindings for Android-specific components, such as activities and fragments.
- **Koin** (<https://insert-koin.io/>): Koin is a lightweight dependency injection library for Kotlin that focuses on simplicity and ease of use. It uses a **domain-specific language (DSL)** to define the dependencies and provide them to the app's components, which makes it easier to do the setup and start using it.

Ultimately, the choice of dependency injection library depends on your specific requirements and preferences, and both Dagger and

Koin are worth considering, depending on your needs. In this case, we are going to use Hilt as it is the current recommendation by Google.

To set up Hilt in our project, follow these steps:

1. Add the Hilt Gradle plugin to your project-level **build.gradle** file (replace **[version]** with the latest version available for you):

```
buildscript {  
    repositories {  
        google()  
    }  
    dependencies {  
        classpath "com.google.dagger:hilt-android-  
            gradle-plugin:[version]"  
    }  
}
```

2. Apply the Hilt Gradle plugin and enable view binding in your app-level **build.gradle** file:

```
apply plugin: 'kotlin-kapt'  
apply plugin: 'dagger.hilt.android.plugin'  
android {  
    ...  
    buildFeatures {  
        viewBinding true  
    }  
}  
dependencies {  
    implementation "com.google.dagger:hilt-  
        android:[version]"  
    kapt "com.google.dagger:hilt-android-  
        compiler:[version]"  
    ...  
}
```

3. Finally, create an **Application** class in our **:app** module. The **Application** class serves as a base class for maintaining the global

application state (this refers to data or settings that need to be maintained throughout the entire life cycle of the application). While it's not created by default, creating a custom **Application** class is crucial for initialization tasks, such as setting up dependency injection frameworks or initializing libraries. In this particular instance, to make Hilt work, you should annotate your **Application** class with the **@HiltAndroidApp** annotation:

```
@HiltAndroidApp
class WhatsPcktApplication : Application() {
    // ...
}
```

With that, we are all set – we will continue defining the modules and dependencies once we advance in this project.

Navigation

The next step is to decide what our approach to handling the navigation between screens and features in our application will be. It is important to note that we are going to use Jetpack Compose to build the user interface of our app, so the chosen approach must be compatible with it.

In this case, we're going to use Navigation Compose as it provides a simple and easy-to-use way to handle in-app navigation within an Android app. Here are some benefits of using Navigation Compose:

- **Declarative UI:** Navigation Compose follows the same declarative approach as Jetpack Compose, which makes it easier to understand and maintain the navigation flow in your application.
- **Type-safety:** With Navigation Compose, you can define your navigation graph and actions in a type-safe way. This helps prevent runtime crashes caused by incorrect navigation action names and arguments.

- **Animation and transition support:** Navigation Compose provides built-in support for animating screen transitions, making it easy to create smooth and visually appealing navigation experiences.
- **Deep linking:** Navigation Compose supports deep linking, allowing you to create URLs that can directly navigate to specific screens or actions within your app. This is useful for implementing features such as app shortcuts, notifications, or sharing content.
- **Integration with Jetpack Compose:** As part of the Jetpack Compose family, Navigation Compose works seamlessly with other Compose libraries and components, allowing you to build a consistent UI and navigation experience across your app.
- **Modularity and scalability:** Navigation Compose enables you to build modular navigation graphs, making it easier to scale your app and manage complex navigation flows.

In summary, Navigation Compose simplifies navigation management, improves the robustness of our app, and will help us to create a more consistent, accessible, and visually appealing user experience.

To start using Navigation Compose, we must do the following:

1. First, we need to include the dependencies that are required in our Gradle files:

```
dependencies {  
    implementation "androidx.navigation:navigation-  
compose:2.5.3"  
}
```

NOTE

The version used in the previous code is the latest stable one at the time of writing this book, but there will likely be a new version by the time you are reading this.

2. Next, in the **app** module, create a new package called **ui.navigation**. Then, create a file called **WhatsPacktNavigation**.
3. Now, create a **NavHost** composable and provide a **NavController** instance. The **NavHost** composable functions as a container for managing navigation between different composables in an app. It acts as the central hub where navigation routes are defined and composables are switched in and out based on the navigation state. Each screen or view in your application corresponds to a composable that **NavHost** can display. Here, we will start by creating the **WhatsPacktNavigation** composable function. This will be responsible for holding **NavHost**:

```
import androidx.compose.runtime.Composable
import androidx.navigation.compose.NavHost
import
androidx.navigation.compose.rememberNavController
@Composable
fun WhatsPacktNavigation() {
    val navController = rememberNavController()
    NavHost(navController = navController,
        startDestination = "start_screen") {
        // Add composable destinations here
    }
}
```

4. Once we've created the first screen (which we'll call **MainScreen**), we will complete **NavHost**, as follows:

```
NavHost(navController = navController,
    startDestination = "start_screen") {
    composable("start_screen") {
        MainScreen(navController) }
}
```

5. We can also include dynamic parameters in the route, like so:

```
NavHost(
```

```
navController = navController,
startDestination = "start_screen"
) {
    composable("start_screen") {
        MainScreen(navController) }
    composable("chat/{chatId}") { backStackEntry ->
        val chatId =
            backStackEntry.arguments?.getString(
                "chatId")
        ChatScreen(navController, chatId)
    }
}
```

Here, we have a second composable that defines another navigation destination associated with the **"chat/{chatId}"** route. The **{chatId}** part is a dynamic parameter that can be passed when navigating to this destination.

Using these two configurations – that is, navigation with and without parameters – should have us covered but since we are using feature-based modularization, we might encounter the problem of having to navigate from one module to another where there isn't a direct dependency between them. In those cases, we will use deep links.

Deep linking allows users to navigate to specific screens or actions within your app using URLs. When defining your composable destinations within **NavHost**, you need to add a **deepLink** parameter with the URI pattern you want to use for that destination. This pattern should include a scheme, a host, and an optional path. In our example, if we have **ChatScreen**, which takes a **chatId** argument, we can add a deep link **URI** like this:

```
NavHost(
    navController = navController,
    startDestination = "start_screen")
{
    composable("start_screen") { MainScreen(navController)
```



```

    }
    composable(
        route = "chat?id={id}",
        deepLinks = listOf(navDeepLink { uriPattern =
            "whatspackt://chat/{id}" })
    ) { backStackEntry ->
        ChatScreen(
            navController,
            backStackEntry.arguments?.getString("id"))
    }
}

```

One common practice to keep our **NavHost** leaner and delegate the definition of routes and URIs to every screen is to define the route with constants. Here is an example:

```

@Composable
fun ChatScreen(
    ...
) {
    object {
        val uri = "whatspackt://chat/{id}"
        val name = "chat?id={id}"
    }
}

```

By doing this, developers can easily manage, update, and maintain the routes in a centralized manner.

Then, in **NavHost**, we would define **uriPattern** using these constants:

```

composable(
    route = NavRoutes.Chat,
    arguments = listOf(
        navArgument(NavRoutes.ChatArgs.ChatId) {
            type = NavType.StringType
        }
    )
)

```

```

    )
    ) { backStackEntry ->
        val chatId = backStackEntry.arguments?.getString(
            NavRoutes.ChatArgs.ChatId)
        ChatScreen(chatId = chatId, onBack = {
            navController.popBackStack() })
    }

```

Instead of adding this information to every screen, a better option is to create a class where we are going to put all the route constants:

```

object NavRoutes {
    const val ConversationsList = "conversations_list"
    const val NewConversation = "create_conversation"
    const val Chat = "chat/{chatId}"
    object ChatArgs {
        const val ChatId = "chatId"
    }
}

```

Having the route's definition in the same place will facilitate reading and maintaining our code so that we can easily manage, update, and maintain the routes in a centralized manner, while also improving code readability and reducing the possibility of errors caused by hard-coded or duplicated strings throughout the code base.

We will place the file that contains this class in our **:common:framework** module as we will need to access those constants from every feature module. Another common practice is to create a dedicated **:common:navigation** module and add the definition of the route and even the **NavHost** definition there. In our case, we will define the routes using the latest approach – that is, route constants:

```

package com.packt.whatspackt.ui.navigation
import androidx.compose.runtime.Composable
import androidx.navigation.NavGraphBuilder

```

```
import androidx.navigation.NavHostController
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import com.packt.feature.chat.ui.ChatScreen
import androidx.navigation.navArgument
import com.packt.framework.navigation.NavRoutes
@Composable
fun MainNavigation(navController: NavHostController) {
    NavHost(
        navController,
        startDestination = NavRoutes.ConversationsList
    ) {
        addConversationsList(navController)
        addNewConversation(navController)
        addChat(navController)
    }
}
```

In the preceding code, we completed our **NavHost** definition.

In our app, we will want to navigate to three different parts of the app (conversations list, create new chat, and the single chat screen). The navigation destinations can be added to **NavHost** by using extension functions on **NavGraphBuilder**. These extension functions are defined as follows:

```
private fun NavGraphBuilder.addConversationsList(
    navController: NavHostController
) {
    composable(NavRoutes.ConversationsList) {
        ConversationsListScreen(
            onNewConversationClick = {
                navController.navigate(
                    NavRoutes.NewConversation
                )
            },
            onConversationClick = { chatId ->
                navController.navigate(
```

```

        NavRoutes.Chat.replace("{chatId}", chatId))
    }
}
)
}
}
private fun NavGraphBuilder.addNewConversation(
navController: NavHostController) {
    composable(NavRoutes.NewConversation) {
        CreateConversationScreen(onCreateConversation = {
            navController.navigate(NavRoutes.Chat)
        })
    }
}
private fun NavGraphBuilder.addChat(navController:
NavHostController) {
    composable(
        route = NavRoutes.Chat,
        arguments = listOf(navArgument(
            NavRoutes.ChatArgs.ChatId) {
                type = NavType.StringType
            })
    ) { backStackEntry ->
        val chatId = backStackEntry.arguments?.getString(
            NavRoutes.ChatArgs.ChatId)
        ChatScreen(chatId = chatId, onBack = {
            navController.popBackStack() })
    }
}
}

```

Here, **addConversationsList(navController)** sets up **ConversationsListScreen** and defines click listeners for navigating to the **NewConversation** and **Chat** destinations.

Then, **addNewConversation(navController)** sets up **CreateConversationScreen** and defines a click listener for navigating to the **Chat** destination upon creating a new conversation.

Finally, **addChat(navController)** sets up **ChatScreen** and extracts the **chatId** argument from **backStackEntry**. It also defines a click listener

for navigating back to the previous screen using `NavController.popBackStack()`.

Now, we are almost ready to hit the **Run** button for the first time. But first, to avoid compilation problems, we should create the screen's composables in their respective modules:

- **ConversationsListScreen** in `:feature:conversations`
- **CreateConversationScreen** in `:feature:create_chat`
- **ChatScreen** in `:feature:chat`

For example, we can create **ChatScreen** and leave it as follows:

```
package com.packt.feature.chat.ui
import androidx.compose.runtime.Composable
@Composable
fun ChatScreen(
    chatId: String?,
    onBack: () -> Unit
) {
}
```

We are missing one last change (for now). We need to include the **MainNavigation** composable as the content in **MainActivity**:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            WhatsPacktTheme {
                val navHostController =
                    rememberNavController()
                MainNavigation(navController =
                    navHostController)
            }
        }
    }
}
```

```
}  
}
```

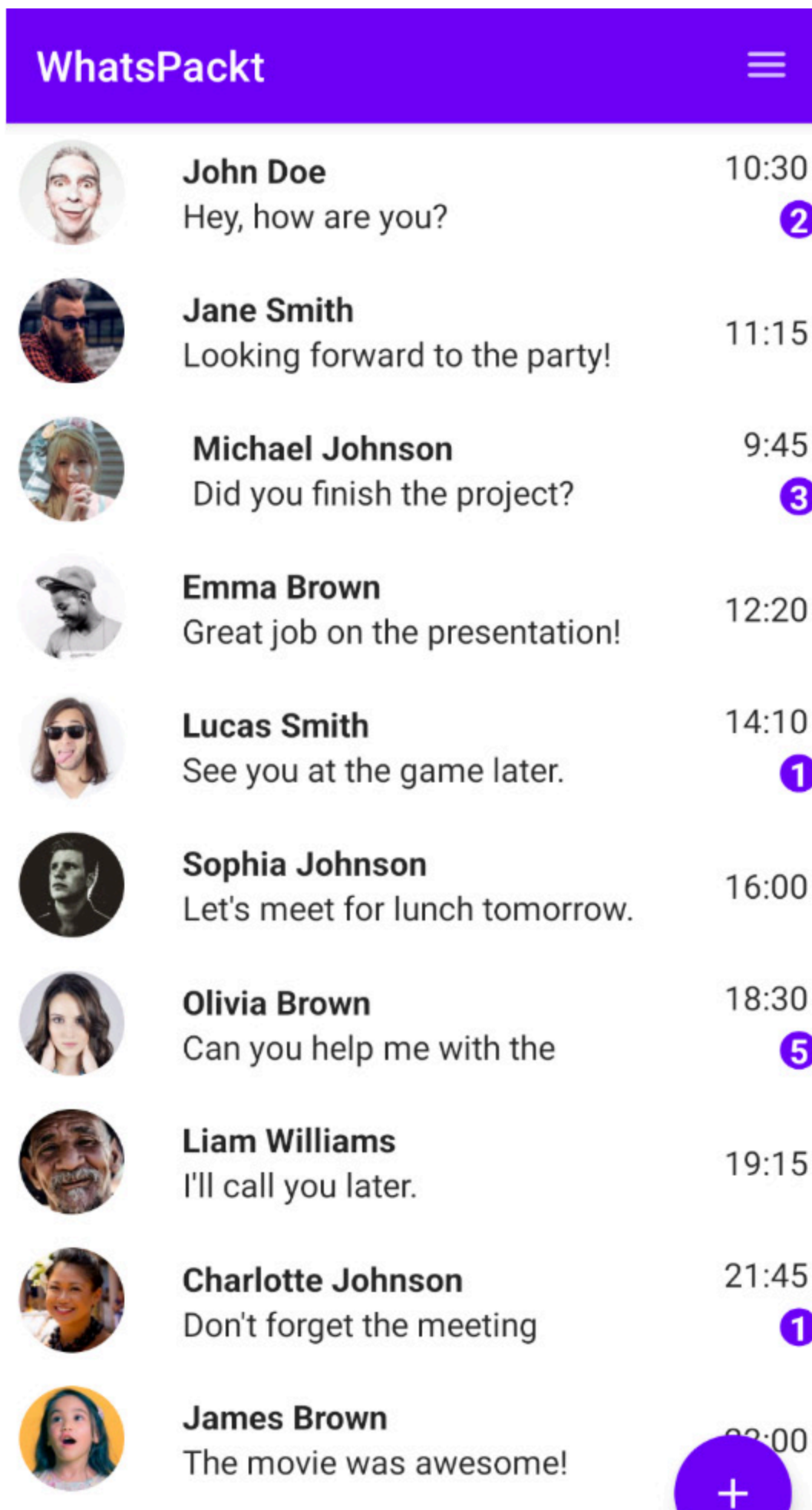
As we can see, we've added **navHostController**, which we created using **rememberNavController()**. This is used to remember the navigation state across recompositions. Here, **navHostController** manages the navigation between different composables in the application. Then, the **MainNavigation** composable is called with **navHostController**.

So far, we have chosen and created the module structure for our app, chosen a dependency injection framework, added the dependencies we need, and structured the navigation that defines how our screens are going to be accessed. Now, it's time for us to start working on each of the screens we need to build this app.

Building the main screen

Now that we have the main structure of our app ready, it is time to start building the main screen.

Let's analyze what components our main screen will have:



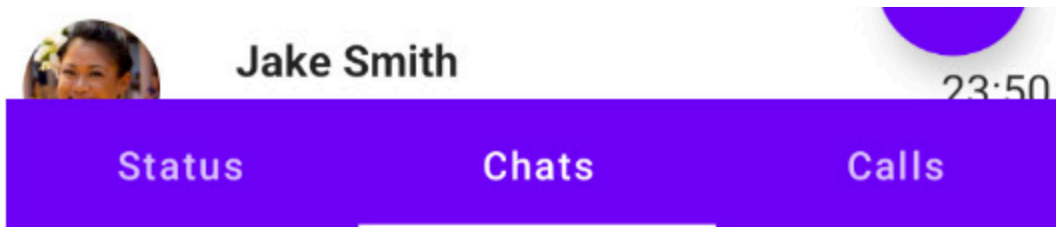


Figure 1.9: The ConversationsList screen

As you can see, we are going to include the following:

- A top bar
- A tab bar to navigate to the main sections (note that this book will only cover the development of the chat section; we will not cover the status and calls sections)
- A list containing the current conversations (which we will complete later in this chapter)
- A floating button to create a new chat

Let's start with the main screen.

Adding a scaffold to the main screen

Previously, we created an empty version of the first screen (**ConversationsListScreen**), as follows:

```
package com.packt.feature.conversations.ui
import androidx.compose.runtime.Composable
@Composable
fun ConversationsListScreen(
    onNewConversationClick: () -> Unit,
    onConversationClick: (chatId: String) -> Unit
) {
    // We will add here the ConversationsListScreen components
}
```

Now, it's time to start working on this screen. The first component we are going to add is a **scaffold**. This is a composable that provides a basic structure for your app's UI. It helps you create a consistent layout

by providing slots for common UI elements, such as a top app bar, a bottom app bar, a navigation drawer, a floating action button, and content. By using **Scaffold**, you can easily organize your app's layout and maintain a consistent look and feel across different screens.

Here's a brief overview of the main components of **Scaffold**:

- **topBar**: A slot for placing a top app bar, typically used for displaying the app's title and navigation icons. You can use the **TopAppBar** composable to create a top app bar.
- **bottomBar**: A slot for placing a bottom app bar, typically used for actions, navigation tabs, or a bottom navigation bar. You can use the **BottomAppBar** or **TabRow** composable to create a bottom app bar.
- **drawerContent**: A slot for placing a navigation drawer, which is a panel that displays the app's navigation options. You can use the **Drawer** or **ModalDrawer** composable to create a navigation drawer.
- **floatingActionButton**: A slot for placing a floating action button, which is a circular button that hovers above the content and represents the primary action of the screen. You can use the **FloatingActionButton** composable to create a floating action button.
- **content**: A slot for placing the main content of the screen, which can be any composable that displays the app's data or UI elements.

In our case, we are going to use **topBar**, **bottomBar** with **TabRow** (to navigate between tabs), **floatingActionButton** (to create new chats), and the content, where we are going to place our main content – in our case, the list of conversations.

Let's create the **Scaffold** composable in our **ConversationsListScreen**. We will add the modifiers for all the components we want to include, but we'll leave them empty (for now):

```

@Composable
fun ConversationsListScreen(
    onNewConversationClick: () -> Unit,
    onConversationClick: (chatId: String) -> Unit
) {
    Scaffold(
        topBar = { /* TopAppBar code */ },
        bottomBar = { /* TabRow code */ },
        floatingActionButton =
            { /* FloatingActionButton code */ }
    ) {
        /* Content code */
    }
}

```

The **Scaffold** composable we have created includes **topBar**, **bottomBar**, **floatingActionButton**, and the content of the main area of the screen. We will continue implementing each of those components.

Now, depending on your Android Studio version, you may see the following error:



Figure 1.10: An error with the content padding parameter

This is happening because the **Scaffold** composable provides a padding parameter to the content Lambda. We will need to take this padding into account when we place the inside components since the scaffold could overlap them if we don't. For example, in our case, we must consider the padding because otherwise, our content will be kept behind **bottomBar**. We will use this parameter layer when we build the content.

Now, we will add a **TopAppBar** composable to the **Scaffold** composable.

Adding the TopAppBar composable to the main screen

The **TopAppBar** composable represents a toolbar located at the top of the screen and provides a consistent look and feel across different screens in your app. It typically displays the following elements:

- **Title:** The main text that's displayed in the app bar, usually representing the app's name or the current screen's title
- **Navigation icon:** An optional icon located at the beginning of the app bar, usually used to open a navigation drawer or navigate back into the app
- **Actions:** A set of optional icons or buttons located at the end of the app bar, representing common actions or settings related to the current screen

To add a **TopAppBar** composable, we must create the **conversations_list_title** string in the module's **strings.xml** file.

Then, we are going to create the **TopAppBar** composable while setting the title to **WhatsPackt** and adding **IconButton** with a menu icon. Here, **IconButton** has an **onClick** function where you can define the action to perform when the button is clicked:

```
topBar = {
    TopAppBar(
        title = {
            Text(stringResource(
                R.string.conversations_list_title))
        },
        actions = {
            IconButton(onClick = { /* Menu action */ }) {
                Icon(Icons.Rounded.Menu,
                    contentDescription = "Menu")
            }
        }
    )
},
```

Next, we are going to create a **TabRow** composable.

Adding the TabRow composable to the bottom of the main screen

The **TabRow** composable is a horizontal row of tabs that allows users to navigate between different views or sections within an app. The **TabRow** composable mainly consists of the following elements:

- **Tabs:** A collection of individual **Tab** composables that represent different sections or views within the app. Each **Tab** composable can have a text label, an icon, or both to describe its content.
- **Selected tab indicator:** A visual indicator that highlights the currently selected tab, making it easy for users to understand which section they are viewing.

Before creating the **TabRow** composable, we'll have to provide a list and the tabs it is going to contain:

@Composable

```
fun ConversationsListScreen(
    onNewConversationClick: () -> Unit,
    onConversationClick: (chatId: String) -> Unit
) {
    val tabs = listOf("Status", "Chats", "Calls")
    Scaffold(
        topBar = {
            ...

```

Then, we can add **TabRow**:

```
bottomBar = {
    TabRow(selectedTabIndex = 1) {
        tabs.forEachIndexed { index, tab ->
            Tab(
                text = { Text(tab) },
                selected = index == 1,
                onClick = { /* Navigation action */ }
            )
        }
    }
},

```

For every row, we are adding a **Tab** composable, where we indicate the title (using a **Text** composable), the selected value when the tab is selected, and the **onClick** action (which we are not implementing).

After that, we can make our code more readable by creating a data class to store the title of the **Tab** composable:

```
data class ConversationsListTab(
    @StringRes val title: Int
)
fun generateTabs(): List<ConversationsListTab> {
    return listOf(
        ConversationsListTab(
            title = R.string.conversations_tab_status_title

```

```
    ),  
    ConversationsListTab(  
        title = R.string.conversations_tab_chats_title  
    ),  
    ConversationsListTab(  
        title = R.string.conversations_tab_calls_title  
    ),  
)  
}
```

Then, we can change our **TabRow** code:

```
bottomBar = {  
    TabRow(selectedTabIndex = 1) {  
        tabs.forEachIndexed { index, _ ->  
            Tab(  
                text = { Text(stringResource(  
                    tabs[index].title)) },  
                selected = index == 1,  
                onClick = {  
                    // Navigate to every tab content  
                }  
            )  
        }  
    }  
}
```

TabRow composables are usually combined with a pager, where the content will be shown. When clicking and navigating between tabs, the main content that's displayed should change.

Now, let's add the pager to our screen content.

Adding a pager

A pager is a UI component that allows users to swipe through multiple pages or screens horizontally or vertically. It is commonly used to display screens or views in a carousel-like fashion.

We are going to use **HorizontalPager**, which, as its name suggests, allows the user to horizontally swipe between screens or composables. One of its main advantages is that it will not create all pages at once; it will only create the current page and the immediate previous/next pages, which will be off-screen. Once a page is out of this three-page window, it will be removed.

To do so, we are going to have to tweak some of the previous code we had in our **Conversations ListScreen** composable:

```
@OptIn(ExperimentalFoundationApi::class)
@Composable
fun ConversationsListScreen(
    onNewConversationClick: () -> Unit,
    onConversationClick: (chatId: String) -> Unit
) {
    val tabs = generateTabs()
    val selectedIndex = remember { mutableStateOf(1) }
    val pagerState = rememberPagerState(initialPage = 1)
    ...
}
```

First, since **HorizontalPager** is part of the foundation API and is (at the time of writing) an experimental API (which means that it could change its public interface in the future), we need to add the **@OptIn(ExperimentalFoundationApi::class)** annotation.

Second, we have added a new field called **pagerState**. Its responsibility is to hold the state of the pager, including information about the number of pages, the current page, the scrolling position, and the scrolling behavior.

Next, we will add **HorizontalPager** to the content function, as follows:

```
content = { innerPadding ->
```

```

HorizontalPager(
    modifier = Modifier.padding(innerPadding),
    pageCount = tabs.size,
    state = pagerState
) { index ->
    when (index) {
        0 -> {
            //Status
        }
        1 -> {
            ConversationList(
                conversations = emptyList(),
                onConversationClick = onConversationClick
            )
        }
        2 -> {
            // Calls
        }
    }
}

LaunchedEffect(selectedIndex.value) {
    pagerState.animateScrollToPage(selectedIndex.value)
}
}

```

Here, we will be using a **LaunchedEffect** function. This function is used to manage side effects, such as launching tasks that have been completed asynchronously in the context of a composable hierarchy. Side effects are operations that can have an impact outside of the composable function itself, such as network requests, database operations, or, in the case of the previous example, scrolling to a specific page in a pager.

LaunchedEffect takes a key (or a set of keys) as its first parameter. When the key changes, the effect will be re-launched, canceling any ongoing work from the previous effect. The second parameter is a suspending Lambda function, which will be executed in the effect's coroutine scope.

The main advantage of using **LaunchedEffect** is that it integrates well with the Compose life cycle. When the composable that called **LaunchedEffect** leaves the composition, the effect will be automatically canceled, cleaning up any ongoing work.

Coming back to our code, in our case, we are changing the current page in **pagerState** and animating the scroll to the next selected page. This will be triggered every time **selectedIndex.value** is changed.

The next component will allow the user to create a new chat – we will create this button using a **FloatingActionButton** composable.

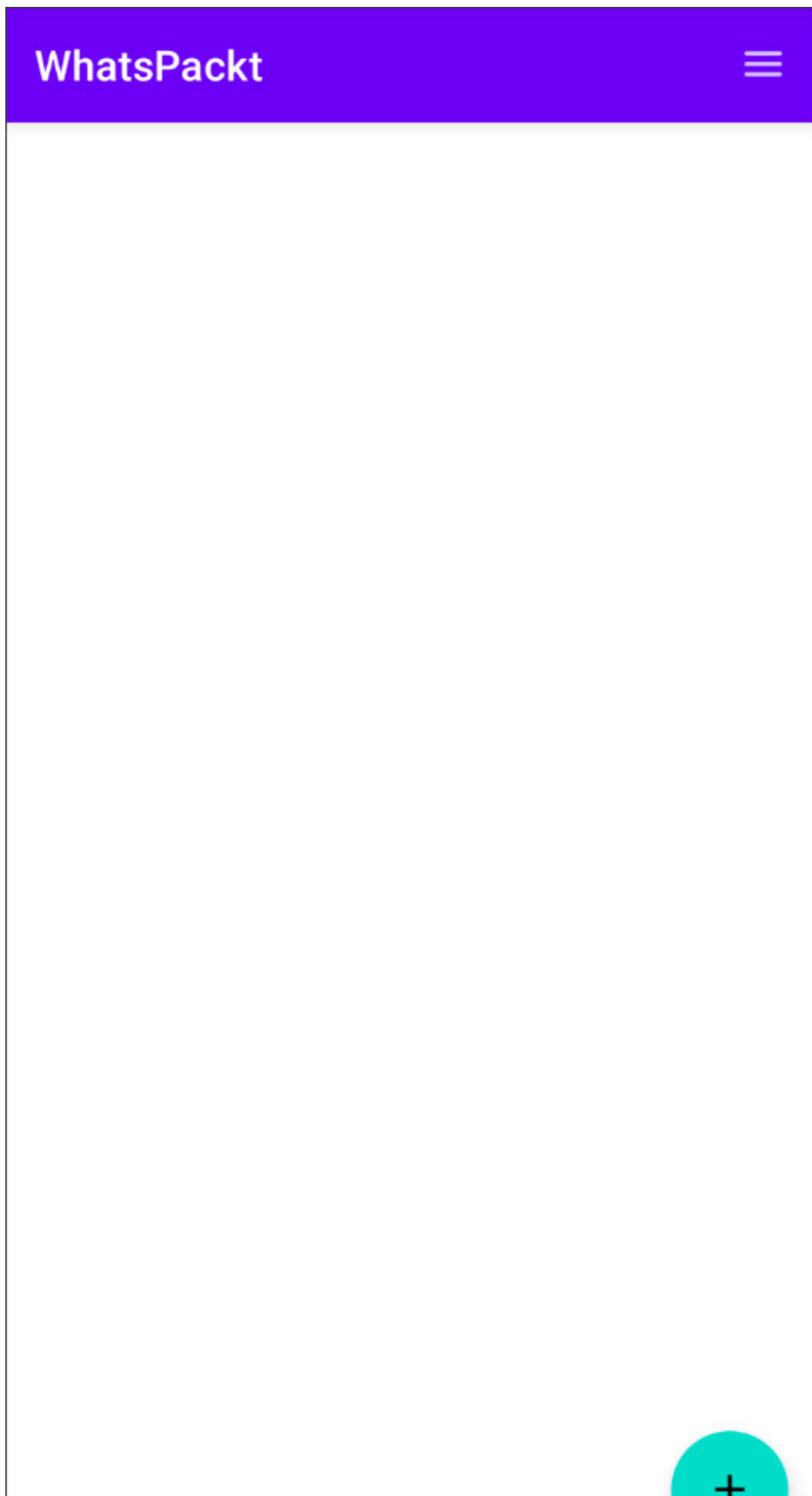
Adding the FloatingActionButton composable

The **FloatingActionButton** composable is a Material Design composable that represents a circular button floating above the UI. It's typically used to promote the primary action in an application (for example, adding a new item, composing a message, or starting a new process). Following the Material Design guidelines (you can check them here: <https://m3.material.io/>), we are going to use it to create a new chat from **ConversationsListScreen**:

```
floatingActionButton = {  
    FloatingActionButton(  
        onClick = { onNewConversationClick() }  
    ) {  
        Icon(  
            imageVector = Icons.Default.Add,  
            contentDescription = "Add"  
        )  
    }  
}
```

Our **FloatingActionButton** composable is taking an **onClick** modifier. Here, we will include the code to navigate to the create chat screen. Inside this button, we have included an **Icon** composable, which we are using as an image of one of the **Icons.Default** predefined images.

At this point, our conversations list screen should look similar to this:



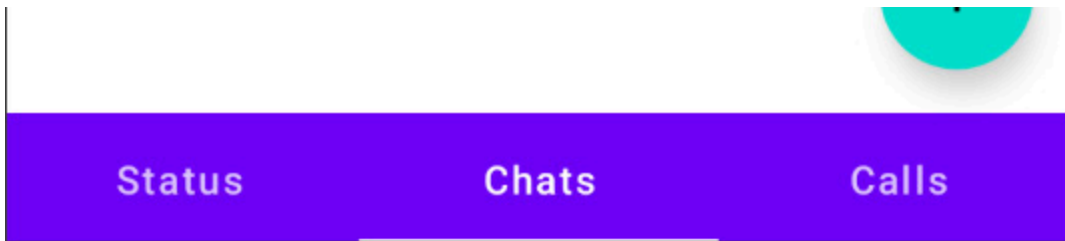


Figure 1.11: The conversations list screen with a top bar, tab bar, and floating action button

With that, we have created our **Scaffold** composable with all the elements we need to help the user navigate. Now, we are ready for the last step (and the most important one) to complete the screen: creating the list of existing conversations. To do that, we are going to start creating a conversation item.

Creating the conversations list

In this section, we are going to create all the pieces we need to show the conversations list. We will start with the UI data model, which will represent the information that the app is going to show in the list, the **Conversation** composable, which will draw every item of the list, and finally the list composable itself.

Modeling the conversation

First, we are going to model what is going to be the entity we will be using through our conversations list components: the **Conversation** model.

As part of the conversation model, we want to show the avatar of the other participant (we are just doing one-to-one conversations), their name, the first line of the last message, the time the message was received, and a number indicating how many unread messages there are.

Taking that information into account, we will start creating a data class to hold the data we'll need:

```
data class Conversation(  
    val id: String,  
    val name: String,  
    val message: String,  
    val timestamp: String,  
    val unreadCount: Int,  
    val avatar: String  
)
```

As the avatar could be reusable across the app, we are going to create it first. We can include it in the `:common:framework` module so that it is visible and can be reused from the other feature modules.

Jetpack Compose doesn't include support to asynchronously load images from a URL out of the box, but there are plenty of third-party libraries that will help us accomplish this. The most popular options are Coil and Glide, which, in terms of performance, caching, and image loading, are quite similar. We are going to use Coil just for simplicity and because it is Kotlin-first (whereas Glide is programmed in Java).

As always, we need to include the dependency in our module's `build.gradle` file:

```
dependencies {  
    ...  
    implementation "io.coil-kt:coil-compose:${latest_version}"  
    ...  
}
```

At this point, we're ready to create our **Avatar** composable:

```

@Composable
fun Avatar(
    modifier: Modifier = Modifier,
    imageUrl: String,
    size: Dp,
    contentDescription: String? = "User avatar"
) {
    AsyncImage(
        model = imageUrl,
        contentDescription = contentDescription,
        modifier = modifier
            .size(size)
            .clip(CircleShape),
        contentScale = ContentScale.Crop
    )
}

```

Here, we are creating an avatar using **AsyncImage**, which will load an image provided by a URL. This image will be modified to have a circular shape. Also, we should pass the size of the image when using this composable (we have chosen 50 density-independent pixels).

Now, we can create **ConversationItem**:

```

import androidx.compose.foundation.layout.*
import androidx.compose.material.MaterialTheme
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.packt.feature.conversations.ui.model.Conversation
import com.packt.framework.ui.Avatar
@Composable
fun ConversationItem(conversation: Conversation) {
    Row(

```

```

        modifier = Modifier
            .fillMaxWidth()
            .padding(8.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Avatar(
            imageUrl = conversation.avatar,
            size = 50.dp,
            contentDescription =
                "${conversation.name}'s avatar"
        )
        Spacer(modifier = Modifier.width(8.dp))
        Column {
            Text(
                text = conversation.name,
                fontWeight = FontWeight.Bold,
                modifier = Modifier.fillMaxWidth(0.7f)
            )
            Text(text = conversation.message)
        }
        Spacer(modifier = Modifier.width(8.dp))
        Column(horizontalAlignment = Alignment.End) {
            Text(text = conversation.timestamp)
            if (conversation.unreadCount > 0) {
                Text(
                    text =
                        conversation.unreadCount.toString(),
                    color = MaterialTheme.colors.primary,
                    modifier = Modifier.padding(top = 4.dp)
                )
            }
        }
    }
}

```

Let's take a closer look at what this composable does:

- The **ConversationItem** composable accepts the following parameters: name, message, timestamp, **avatarUrl**, and an optional **unreadMessages** with a default value of 0.

- A **Row** layout is used to arrange the contents horizontally. It has a **fillMaxWidth()** modifier to occupy the full width of the parent and a padding of 8 density-independent pixels.
- The **Avatar** composable is used to display the avatar. We already know how it works – the only thing to remark on is that we want it to have a size of 50 density-independent pixels.
- A **Spacer** composable with a width of 8 density pixels is added to provide some space between the avatar and the text content.
- A **Column** layout is used to arrange the name and message text vertically. The **Column** layout has a **Modifier.weight()** modifier, which ensures that it takes up all the available space between the avatar and the timestamp.
- Inside the **Column** layout, a **Text** composable is used to display the name with a bold font weight and a font size of 16 scale-independent pixels. Another **Text** composable is used to display the message with a maximum of one line and an ellipsis overflow.
- Another **Column** layout is added to the main **Row** layout to arrange the timestamp and unread messages badge vertically. The **Column** layout has a **horizontalAlignment** value of **Alignment.End** to align its children to the end of the available space.
- Inside this second **Column** layout, a **Text** composable is used to display the time with a font size of 12 scale-independent pixels and a gray color.
- A conditional statement checks if there are any unread messages (that is, **conversation.unreadMessages > 0**). If there are unread messages, the unread messages count shows a text with a circular background drawn using the **drawBehind** modifier.

Now that we have our **ConversationItem** composable, it is time to finish this screen. Let's create the **ConversationList** composable!

Creating the ConversationList composable

As the last step for this screen, we are going to create the list of conversations:

```
@Composable
fun ConversationList(conversations: List<Conversation>) {
    LazyColumn {
        items(conversations) { conversation ->
            ConversationItem(
                conversation = conversation
            )
        }
    }
}
```

The **ConversationList** composable takes a list of **Conversation** objects and uses **LazyColumn** to display them efficiently. The **items** function is used to loop through the conversations list and will render **ConversationItem** for each conversation.

Finally, we will include the list in the **HorizontalPager** logic, in our **ConversationsListScreen** composable:

```
HorizontalPager(
    modifier = Modifier.padding(innerPadding),
    pageCount = tabs.size,
    state = pagerState
) { index ->
    when (index) {
        0 -> {
            //Status
        }
        1 -> {
            ConversationList(
                conversations = emptyList(), // Leaving the
                                              list empty
                                              for now
            )
        }
    }
}
```

```

        onConversationClick = onConversationClick
    )
}
2-> {
    // Calls
}
}
}

```

If we want to test it, we can fake the data of the conversations:

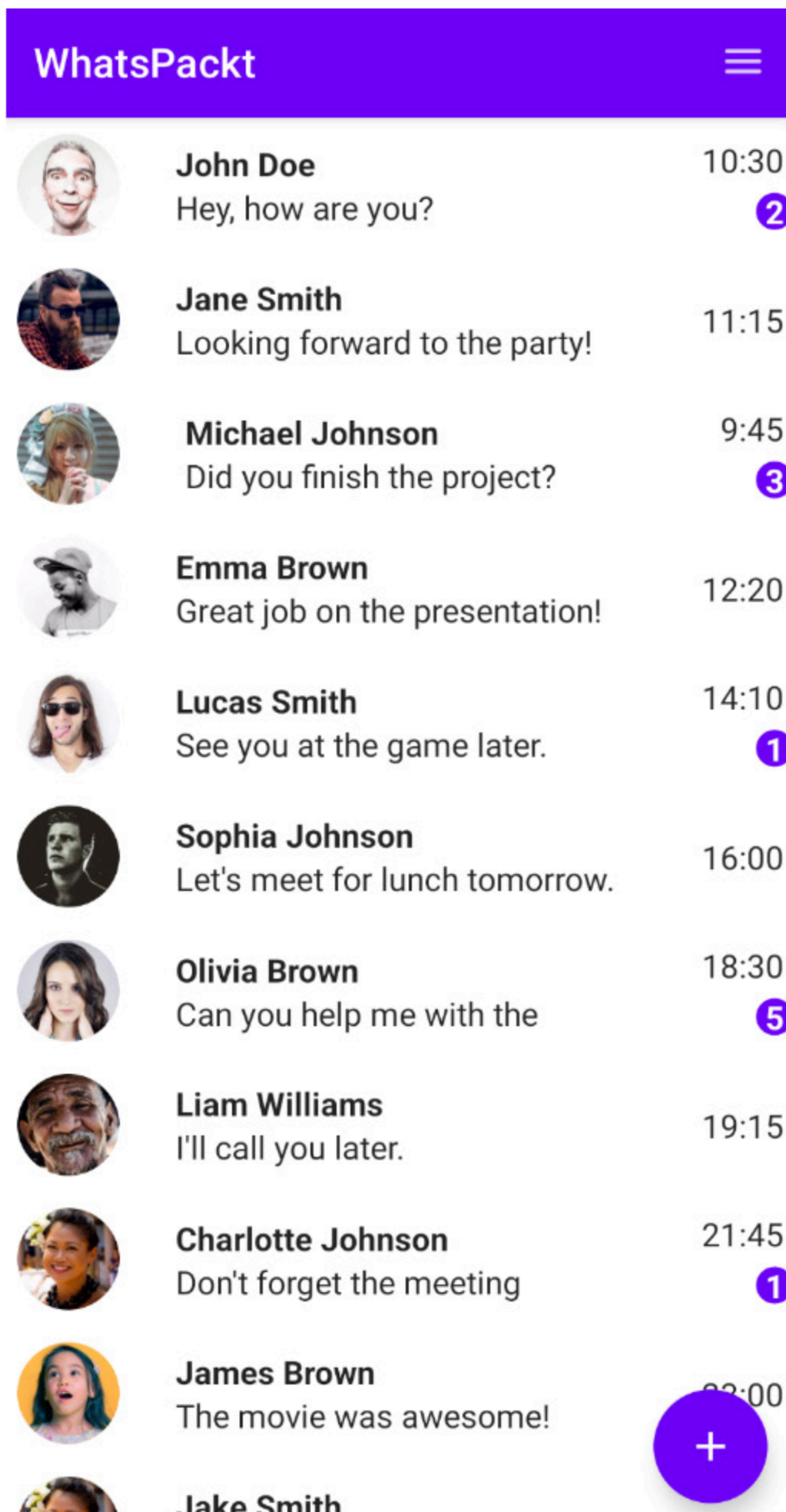
```

fun generateFakeConversations(): List<Conversation> {
    return listOf(
        Conversation(
            id = "1",
            name = "John Doe",
            message = "Hey, how are you?",
            timestamp = "10:30",
            avatar = "https://i.pravatar.cc/150?u=1",
            unreadCount = 2
        ),
        Conversation(
            id = "2",
            name = "Jane Smith",
            message = "Looking forward to the party!",
            timestamp = "11:15",
            avatar = "https://i.pravatar.cc/150?u=2"
        ),
        //Add more conversations here
    )
}

```

Note that here, I'm using a random avatar generator just to make it as similar as possible to how it would be when we connect this UI with real conversations.

The following screenshot shows what our app would look like with more conversations:



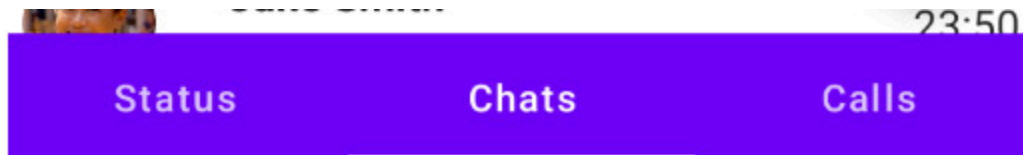


Figure 1.12: ConversationsList screen completed

Now, let's switch to the chat screen, also known as the messages list. Whereas the conversations list is a list showing all the conversations we have, the messages list will show the list of messages we have with one user (a single chat screen).

Building the messages list

In this section, we are going to create the UI models that are needed to create the chat screen and the messages two users could have exchanged. Then, we will create the **Message** composable, and finally, the rest of the screen, including the list of messages.

Modeling the Chat and Message models

Taking into account the information we have to show on the chat screen, we are going to need two data models: one for the static data related to the conversation (for example, the name of the user we are talking to, their avatar, and so on) and one data model per message. This will be the model for the **Chat** model:

```
data class Chat(  
    val id: String,  
    val name: String,  
    val avatar: String  
)
```

In this case, we will need the ID of the chat, the name of the person we are talking to, and their avatar address.

Regarding the **Message** model, we will create the following classes:

```
data class Message(  
    val id: String,  
    val senderName: String,  
    val senderAvatar: String,  
    val timestamp: String,  
    val isMine: Boolean,  
    val messageContent: MessageContent  
)  
sealed class MessageContent {  
    data class TextMessage(val message: String) :  
        MessageContent()  
    data class ImageMessage(val imageUrl: String,  
        val contentDescription: String) : MessageContent()  
}
```

In this case, we need the sender's name, their avatar, the timestamp, whether the message is mine or not (which we will take into account so that we can arrange the message one to the left or the right), and the content of the message.

Since our application is going to have two types of content (messages and images), we need two different kinds of **MessageContent**. That's the reason it has been modeled as a sealed class. We have two data classes with the data needed for every type of message content.

Now, we need to convert these models into some composables.

Creating the **MessageItem** composable

The **MessageItem** composable is going to draw each of our chat messages.

To start, we will create a **Row** layout. We will set the arrangement of the row contents depending on the message's author:

```
@Composable
fun MessageItem(message: Message) {
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = if (message.isMine)
            Arrangement.End else Arrangement.Start
    ) {
        ...
    }
}
```

Then, inside this row, we are going to place the rest of the components of the message. We will start with the avatar; we will only show the avatar if the message is not from the user:

```
if (!message.isMine) {
    Avatar(
        imageUrl = message.senderAvatar,
        size = 40.dp,
        contentDescription = "${message.senderName}'s
                               avatar"
    )
    Spacer(modifier = Modifier.width(8.dp))
}
```

Then, we are going to add a **Column** layout so that we can arrange the rest of the message information:

```
Column {
    if (message.isMine) {
        Spacer(modifier = Modifier.height(8.dp))
    } else {
        Text(
            text = message.senderName,
            fontWeight = FontWeight.Bold
        )
    }
}
```

```
}
when (val content = message.messageContent) {
    is MessageContent.TextMessage -> {
        Surface(
            shape = RoundedCornerShape(8.dp),
            color = if (message.isMine)
                MaterialTheme.colors.primarySurface else
                MaterialTheme.colors.secondary
        ) {
            Text(
                text = content.message,
                modifier = Modifier.padding(8.dp),
                color = if (message.isMine)
                    MaterialTheme.colors.onPrimary else
                    Color.White
            )
        }
    }
    is MessageContent.ImageMessage -> {
        AsyncImage(
            model = content.imageUrl,
            contentDescription =
                content.contentDescription,
            modifier = Modifier
                .size(40.dp)
                .clip(CircleShape),
            contentScale = ContentScale.Crop
        )
    }
}
Text(
    text = message.timestamp,
    fontSize = 12.sp
)
}
```

The message will include the following information:

- **The name of the sender (if the author is not the current user):**

To know if the message is from the current user, the app will check

if the author of the message is the current user using **if** (**message.isMine**. If this is positive, we will add a **Space** composable; if the message author is not the current user, we will show the **Text** composable and their name.

- **The content:** The app will show a bubble containing text if the content is text. The color of the bubble will depend on whether the sender of the message is the current user and the time the message was created. A timestamp with the date and time of creation will be shown at the bottom of the message.

Now that we have **MessageItem**, it's time to create the rest of the chat screen.

Adding the **TopAppBar** and **BottomRow** composables

As we did for the conversations list, we are going to add the **Scaffold** structure and its **TopAppBar** and **BottomRow** composables to this screen:

```
@Composable
fun ChatScreen(
    chatId: String?,
    onBack: () -> Unit
) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Text(stringResource(
                        R.string.chat_title, "Alice"))
                }
            )
        },
        bottomBar = {
            SendMessageBox()
        }
    )
}
```



```

    ) { paddingValues->
        ListOfMessages(paddingValues = paddingValues)
    }
}

```

Note that we are hardcoding the title of the chat. This is just for pre-view purposes; we'll correct that in the next chapter.

In the case of the bottom bar, we are adding a new composable that will contain **TextField** and the send button needed to send a message. This is what this composable will look like:

```

@Composable
fun SendMessageBox() {
    Box(
        modifier = Modifier
            .defaultMinSize()
            .padding(top = 0.dp, start = 16.dp,
                end = 16.dp,
                bottom = 16.dp)
            .fillMaxWidth()
    ) {
        var text by remember { mutableStateOf("") }
        OutlinedTextField(
            value = text,
            onChange = { newText -> text = newText },
            modifier = Modifier
                .fillMaxWidth(0.85f)
                .align(Alignment.CenterStart)
                .height(56.dp),
        )
        IconButton(
            modifier = Modifier
                .align(Alignment.CenterEnd)
                .height(56.dp),
            onClick = {
                // Send message here
                text = ""
            }
        )
    }
}

```

```
    ) {  
        Icon(  
            imageVector = Icons.Default.Send,  
            tint = MaterialTheme.colors.primary,  
            contentDescription = "Send message"  
        )  
    }  
}  
}
```

Here, we are creating a **Box** composable to arrange the children composables accordingly (the text field at the left and the **Send** button at the right). Then, we're defining a property called **text** to store text field changes and using the **remember** delegate to remember its last value between recompositions.

As shown in the preceding code block, the main components of this composable are as follows:

- **OutlinedTextField**: To write the message. It will take its value from the text property and modify it every time the value of the text field changes.
- **IconButton**: To send the message. Its **onClick** parameter is not doing anything yet (apart from restarting the **text** property value). We will configure this in the next chapter.

With that, our chat screen is almost ready. The last thing we need to do is add the messages list.

Adding the messages list

Earlier, we were adding the messages list as a composable in the **content** parameter of the **Scaffold** composable. This composable will look as follows:

```
@Composable
```

```

fun ListOfMessages(paddingValues: PaddingValues) {
    Box(modifier = Modifier
        .fillMaxSize()
        .padding(paddingValues)) {
        Row(modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp)
        ) {
            LazyColumn(
                modifier = Modifier
                    .fillMaxSize(),
                verticalArrangement =
                    Arrangement.spacedBy(8.dp),
            ) {
                items(getFakeMessages()) { message ->
                    MessageItem(message = message)
                }
            }
        }
    }
}

```

With that, we have added **LazyColumn**, which will show the list – every item in the list is a **MessageItem** composable.

Since we haven't connected it to any kind of data source yet, we are using a function to generate a list of fake messages just for preview purposes:

```

fun getFakeMessages(): List<Message> {
    return listOf(
        Message(
            id = "1",
            senderName = "Alice",
            senderAvatar =
                "https://i.pravatar.cc/300?img=1",
            isMine = false,
            timestamp = "10:00",

```

```
        messageContent = MessageContent.TextMessage(
            message = "Hi, how are you?"
        )
    ),
    Message(
        id = "2",
        senderName = "Lucy",
        senderAvatar =
            "https://i.pravatar.cc/300?img=2",
        isMine = true,
        timestamp = "10:01",
        messageContent = MessageContent.TextMessage(
            message = "I'm good, thank you! And you?"
        )
    ),
)
}
```

You can add more messages if you want by adding them to the list that's been created inside the `getFakeMessages()` function.

Finally, we should have a screen that looks like this:

Chat with Alice



Alice

Hi, how are you?

10:00

I'm good, thank you! And you?

10:01



Alice

Super fine!

10:02

Are you going to the Kotlin conference?

10:02



Alice

Of course! I hope to see you there!

10:03



Alice

I'm going to arrive pretty early

10:03



Alice

So maybe we can have a coffee together

10:03



Alice

Wdyt?

10:03

Figure 1.13: Chat screen UI finished

With that, we are done with the user interface for now. We will continue working on this app during the next two chapters!

Summary

In this first chapter, we started our first project, WhatsPackt, a messaging app.

We accomplished several initial tasks to build this app, such as organizing modules, preparing dependency injection and navigation, constructing the main screen, creating the conversations list, and building the messages list.

Throughout this process, we've learned about modularization and the various approaches to organizing it. We've also learned about popular libraries for managing dependency injection, how to initialize them, and how to set up Compose navigation. Additionally, we became familiar with using Jetpack Compose to create our user interface.

As we move forward, it's time to give some love and life to our chats. In the next chapter, we will explore how to retrieve and send messages and integrate them into our recently created user interfaces.